

Documentacion Figuritas.app

Documentacion Figuritas.app.....	1
1. Backend.....	3
1.1 Dependencias.....	3
Sistemas externos.....	3
The Sports.....	3
Telegram.....	3
Frameworks.....	3
Spring-boot.....	3
Spring-Data MongoDB.....	3
Librerías.....	3
Lombok.....	3
Flapdoodle.....	4
JUnit 5 + Mockito.....	4
Herramientas y otros.....	4
K6 - Test de carga.....	4
Json Web Token.....	4
JaCoCo + SpotBugs.....	4
1.2 Dominio.....	5
1.3 Persistencia.....	6
Paginación.....	6
Objetos Embebidos.....	6
Listas Lazy.....	6
Otras consideraciones.....	7
2. Frontend.....	7
2.1 Dependencias.....	7
Frameworks.....	7
React 19.....	7
Librerías.....	7
React Router 7.....	7
Axios.....	7
Bootstrap.....	7
Herramientas y otros.....	8
Vite 8.....	8
Nginx.....	8
3. Infraestructura.....	8
3.1 Docker.....	8
Docker compose.....	8
3.2 Backend.....	8
3.3 Frontend.....	9
3.4 Compose prod.....	9
3.5 Compose dev.....	10

4. Deploy.....	10
5. Uso de la Inteligencia Artificial.....	10
5.1 Creación de test según casos de uso.....	10
5.2 Diseño de las plantillas para el frontend.....	11
5.3 Generación de Componentes React.....	11
5.4 Generación de Documentación.....	11
5.5 Debugging errores del Front y Back.....	11
5.6 Revisión de Pull Request en Github.....	11
6. Telegram:.....	11
6.1 Decisión de no hacer un servicio aparte:.....	11

1. Backend

1.1 Dependencias

Sistemas externos

The Sports

Al iniciar la app, se lanza un proceso asíncrono que consulta la API externa [TheSportsDB](#) para obtener la URL de la foto de cada jugador y la persiste en la Figurita. Solo procesa las figuritas que todavía no tienen imagen, y retoma las que quedaron colgadas.

Para más especificaciones, revisar el archivo “Enriquecimiento-imagenes-jugadores.md” en la carpeta de documentación.

Telegram

La implementación del bot de telegram se decidió hacerla en el mismo servicio donde trabaja la API por una cuestión de tiempos y no agregar complejidad al sistema. La realidad es que la mejor decisión hubiera sido hacer un servicio aparte, para no agregar responsabilidades a la API que ya existe. La ventaja de haber hecho la implementación que elegimos es que pudimos reutilizar mucha de la lógica ya existente en el servicio.

El estado actual del sistema no permite enviar notificaciones desde el backend hacia el usuario. Hay algunas clases relacionadas a este feature que se decidieron dejar para una futura implementación.

Para más especificaciones, revisar el archivo “TELEGRAM_BOT.md” en la carpeta de documentación.

Frameworks

Spring-boot

Esta dependencia nos brinda un framework de trabajo que el equipo conoce en un nivel bastante alto en comparación con otros.

El framework ofrece automatizar la mayoría de controles HTTP internos, y a su vez, una cantidad de dependencias fácilmente integrables con él, permitiendo una flexibilidad atractiva.

Spring-Data MongoDB

Esta dependencia forma parte del framework Spring Boot, nos permite hacer un mapeo del mundo de Objetos a Documental (y viceversa) empleando tags y consultas a nivel de objetos. Tiene la ventaja de su simplicidad de uso, abstrayendo al desarrollador de aspectos como Schema y mapeo de objetos.

Librerías

Lombok

Lombok es una librería que nos permite **enriquecer de métodos** en las clases sin llenar de código las mismas, permite crear getters, setters, constructores, builder, etc.

Los más usados en el proyecto son:

- **@Getters**
- **@Setters**
- **@Builder**: Muchas clases comenzaron a tener muchos constructores básicamente para desarrollar en los test, los cuales no son empleados en condiciones normales. El builder nos permite no llenar de distintos constructores en las clases.
- **@NoArgsConstructor**: Hace constructores sin parámetros. Usados para el framework de **Spring Data MongoDB**.
- **@AllArgsConstructor**: Hace constructores con todos los parámetros, usados para test cuando se requiere.
- **@RequiredArgsConstructor**: Hace constructores con aquellos atributos con **@NonNull** o **final**. Son usados en **Controladores, Servicios y Repositorios** para el framework **Spring Boot**.

Flapdoodle

Flapdoodle es una librería que simula una base de datos dentro de la memoria pensada para **entornos de testeo**. Con esta herramienta podemos comprobar el correcto funcionamiento de los repositorios y posibles conflictos que al mockear podríamos no detectar.

Esta librería, junto con el framework **Spring Data MongoDB**, nos permitió desarrollar test de integración empleando una clase abstracta **MongoTestBase**.

JUnit 5 + Mockito

Esta librería nos permite desarrollar test de alta fidelidad y permitiendo mocks de ciertos elementos. Se eligió por el fuerte vínculo que presenta con Spring-boot y facilidad de implementación, haciendo que el trabajo de desarrollar test sea rápido y eficiente.

Herramientas y otros

K6 - Test de carga

Se implementó K6 para el desarrollo de testeos de carga para las rutas más pesadas (/sugerencias, /subastas, /intercambios, etc) y ver cómo reacciona nuestro sistema ante carga intensa de usuarios.

Json Web Token

Herramienta empleada para mantener una sesión a lo largo del uso del sistema por un usuario. Permite guardar contenidos a elección encriptados y firmados con un secreto, garantizando que cierta información persista en más de un pedido HTTP.

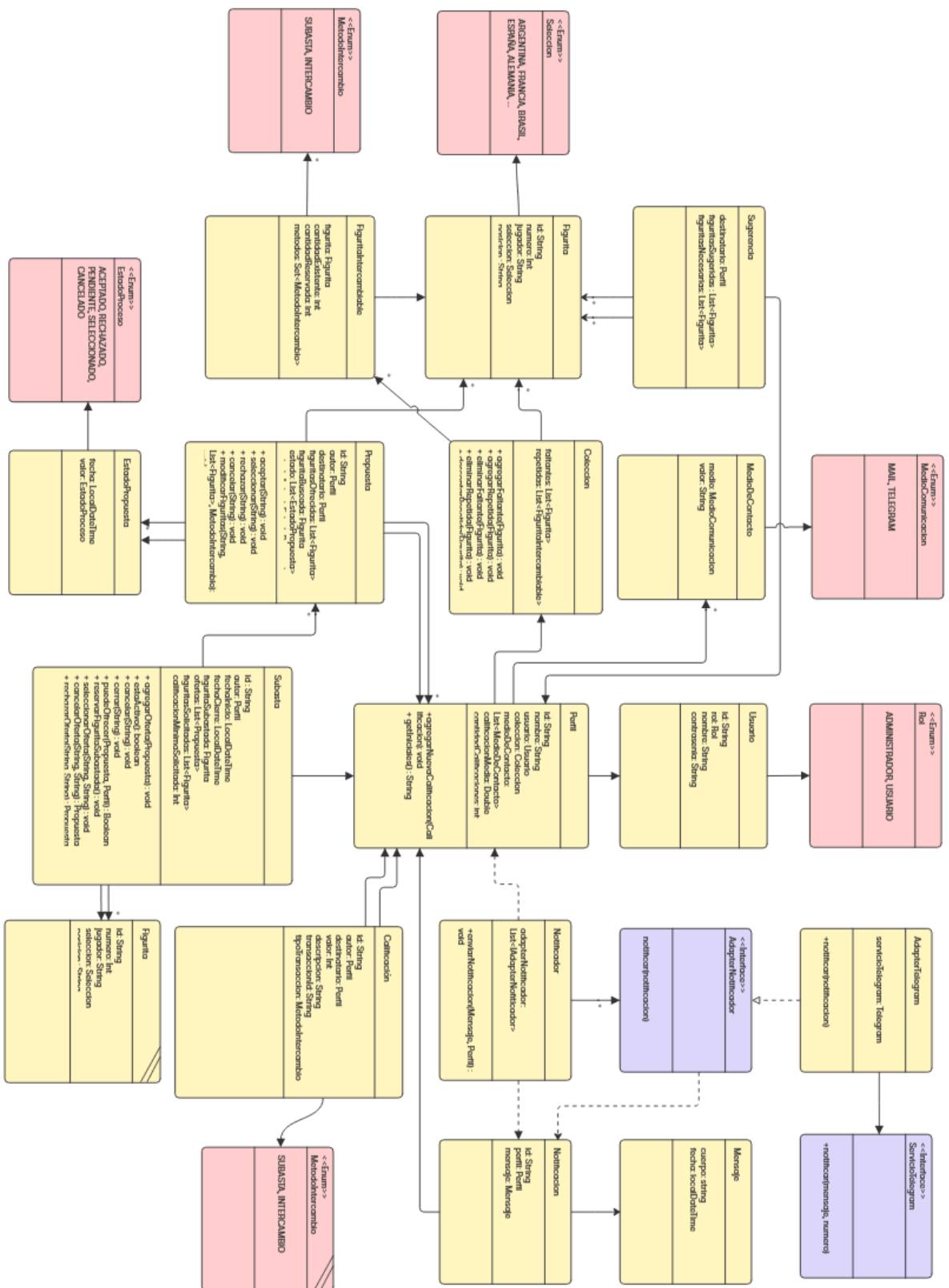
Esta herramienta fue seleccionada debido a su amplio conocimiento en los miembros del equipo, además de permitir tener un control amplio de cada sesión y sus campos.

JaCoCo + SpotBugs

JaCoCo nos brinda el servicio de cobertura de los archivos sumado con SpotBugs. En conjunto nos permite una robustez garantizada del porcentaje que el equipo decidió como válido (80% de paquetes).

JaCoCo nos da una facilidad a la hora de comprobar qué archivos requieren de mayor cobertura en los test, y en consecuencia, aumentar la robustez del sistema en general.

1.2 Dominio



1.3 Persistencia

Para la persistencia se decidió trabajar con **MongoDB**, ya que:

- Nos permite tener dinamismo en el manejo de la información.
- Conservar relaciones y consultas flexibles.
- El equipo tiene un mayor conocimiento en el trabajo con bases de datos documentales.
- Los frameworks seleccionados nos brindan robustez y son ampliamente conocidos en la industria.
- El mapeo de Documental y objetos es semi-directo.

Paginación

Se desarrolló una clase **PaginaResultado<>**, con el objetivo de devolver listas de objetos con atributos comunes como:

- **Data**: Lista de objetos
- **Numero**: Número de página actual
- **CantidadDeElementos**: Resultados totales que cumplen con una query.
- **CantidadDePaginas**: Cantidad de páginas que entran en esa cantidad de elementos, utilizado para saber cuántas páginas quedan hasta llegar al fin de la lista.

Si se desea devolver más información que la mencionada arriba, se pueden componer clases entre ellas, permitiendo un uso común entre ellas.

Objetos Embebidos

En algunas clases determinamos que, para aprovechar mejor el motor de mongoDB, la mejor elección era embeber información dentro de la colección (en lugar de referenciar y/o hacerla una colección aparte). Esto se dio en dos clases particularmente:

- **Subasta.ofertas**: Las ofertas (lista de propuestas) se las embebe dentro de la colección subasta, ya que las ofertas de una subasta solo existen para la subasta en sí y no tienen sentido fuera de ella.
- **Colección.repetidas**: Las repetidas (**FiguritaIntercambiable**) de un perfil, son contenidas en el objeto de colección, y nuevamente solo tienen sentido dentro de ella. Lo que significa que, ninguna **FiguritaIntercambiable** existe fuera de una y solo una colección.

Listas Lazy

Se desarrolló un comportamiento extra en algunas de las clases que implementan listas en sus atributos, diseñada para evitar la carga de elementos cuando algunas acciones no interactúan con ellos. Esto se implementó de dos maneras distintas:

- **Lazy nativo**: Implementado por el framework Spring Data MongoDB.
- **Lazy manual**: En algunos casos, donde las listas son embebidas en la colección, no podemos usar el lazy que proviene del **Tag** (ya que embeber se denota sin tag). Para implementar lazy en estos casos, se decidió crear clases que tengan el objetivo de decirle a la query que campos cargar de ese objeto.

Otras consideraciones

A la hora de desarrollar el sistema de calificación, nos percatamos de que el cálculo del promedio en memoria podría ocasionar problemas en el futuro. Es por ello que se decidió guardar un valor medio y una cantidad de calificaciones asociada a ese perfil.

De esta manera no es necesario cargar todas las calificaciones en memoria (las cuales pueden ser muchas), permitiéndonos una mejor eficiencia de recursos (memoria y tiempo) a cambio de espacio en disco.

2. Frontend

2.1 Dependencias

Frameworks

React 19

Framework principal utilizado para el desarrollo de la interfaz de usuario de la aplicación. Nos permitió construir una SPA (Single Page Application) basada en componentes reutilizables, facilitando la modularización, mantenimiento y escalabilidad del frontend. Además, su ecosistema y compatibilidad con múltiples librerías simplificaron la integración de herramientas complementarias.

Librerías

React Router 7

Librería utilizada para la gestión de rutas dentro de la aplicación. Permitió implementar navegación dinámica entre vistas sin necesidad de recargar la página, mejorando la experiencia del usuario y facilitando la organización de las distintas pantallas del sistema.

Axios

Librería altamente conocida por el equipo, que nos permitió validar errores de peticiones a la API, entre otras cosas.

Bootstrap

Librería de estilos para nuestros componentes de React, en conjunto con estilos configurados por el equipo. Se decidió utilizarla por su facilidad para integrar estilos predeterminados y consistentes entre componentes, manteniendo la aplicación responsive y acelerando el desarrollo visual de la interfaz.

Herramientas y otros

Vite 8

Herramienta de build y entorno de desarrollo utilizada para el frontend. Fue elegida por su rapidez en tiempos de compilación y recarga en caliente (Hot Reload), permitiendo un flujo de desarrollo más ágil y eficiente en comparación con otras alternativas tradicionales.

Nginx

Servidor web utilizado para servir la aplicación frontend en ambientes de producción. Permitió desplegar la aplicación de manera eficiente, manejando archivos estáticos y facilitando la configuración del entorno de despliegue.

3. Infraestructura

3.1 Docker

Para la containerización se decidió trabajar con multi-stage builds tanto en el backend como en el frontend. La imagen final no tiene el compilador ni las dependencias de build, únicamente el ejecutable. Además, cada dockerfile define stages separadas para desarrollo y producción.

Docker compose

Para el desarrollo de manera portable y testeos sin la instalación en el sistema de desarrollo, se empleó la funcionalidad de docker compose. En la misma, describimos los servicios que nuestro sistema incluye de manera local, haciendo esto un ambiente ideal para el desarrollo del mismo.

Esta funcionalidad fue de gran valor para el testeo de carga, haciendo un entorno portable y limpio por ejecución, asegurando la no dependencia entre testeos. Gracias a esto, podemos utilizar la herramienta k6 sin instalarla en las computadoras de los desarrolladores.

3.2 Backend

El Dockerfile del backend define cuatro stages:

- **dependencias:** Parte de maven:3.9-eclipse-temurin-17 y descarga todas las dependencias y plugins con mvn dependency:go-offline. Al separar este paso, Docker cachea la capa de dependencias y evita redescargarlas en cada build mientras el pom.xml no cambie.
- **dev:** Agrega el código fuente sobre la etapa anterior, sin correr tests ni validaciones. Es el stage objetivo para el entorno de desarrollo.

- **build:** Corre mvn verify sobre la etapa dev, ejecutando tests, análisis de Spot Bugs y cobertura mínima con JaCoCo. En CI se puede sustituir por mvn package -DskipTests si los tests ya corrieron en un step previo, vía el argumento MAVEN_ARGS.
- **runtime:** Parte de eclipse-temurin:17-jre-alpine (JRE Alpine, sin JDK ni Maven). Crea un usuario sin privilegios (appuser) y copia únicamente el JAR generado. La plataforma se fuerza a linux/amd64 porque la imagen Alpine de Eclipse Temurin no tiene soporte ARM64 (en mac puede fallar, en windows es transparente). Es la imagen final utilizada en producción.

3.3 Frontend

El Dockerfile del frontend define cuatro stages:

- **deps** Parte de node:22-alpine y corre npm ci para instalar los paquetes. Al separar este paso, Docker cachea node_modules mientras package-lock.json no cambie.
- **dev:** Agrega el código fuente sobre el stage anterior sin compilar. Es el stage objetivo para el entorno de desarrollo.
- **builder:** Corre npm run build sobre el stage dev, generando los assets estáticos en /app/dist.
- **production:** Parte de nginxinc/nginx-unprivileged:alpine, que corre como usuario no root en el puerto 8080. Copia el directorio /dist generado por la etapa anterior y la configuración personalizada de Nginx. Es la imagen final utilizada en producción

3.4 Compose prod

El archivo docker-compose.yml orquesta dos servicios: backend y frontend. Para garantizar que las primeras llamadas a la API no fallen durante el arranque, el frontend declara depends_on con condition: service_healthy sobre el backend. El backend expone /ping como endpoint de salud, con el siguiente health check:

- Intervalo: 10s
- Timeout: 5s
- Reintentos: 6
- Período inicial: 15s (margen para que Spring Boot inicialice)

Ambos servicios tienen restart: unless-stopped para que Docker los recupere automáticamente ante una caída.

3.5 Compose dev

El archivo `docker-compose.dev.yml` incorpora un servicio adicional de MongoDB (mongo:7) con réplica set, necesario para soportar transacciones. El proceso de inicialización del replica set se delega a un contenedor efímero (mongo-init) que corre `rs.initiate()` si el replica set aún no fue configurado, ignorando la operación en arranques posteriores.

El backend buildea hasta el stage dev y corre con `mvn spring-boot:run`. El frontend buildea hasta el stage dev y corre con `npm run dev --host`, habilitando el servidor de Vite en modo red.

Los cambios en el código fuente del frontend se sincronizan al contenedor mediante Compose Watch, que transfiere archivos sin requerir volume mounts. Vite detecta los cambios y actualiza el browser automáticamente (Hot Reload). Los cambios en el backend requieren un redesplicue manual con `make dev-back`.

3.6 Compose test

El archivo `docker-compose.test.yml` incorpora la funcionalidad de probar el código mediante K6, permitiendo la ejecución portable sin la descarga en la PC de desarrollo de la herramienta. El mismo, pone en contexto al sistema para realizar una tarea de pruebas de carga, desactivando la interacción con sistemas externos.

4. Deploy

Para el deploy, se eligió el servicio de **Render**, el cual provee una interfaz amigable para el uso de docker. Se lanzaron dos servicios web: [figunet](#) (frontend) y [figunet-api](#) (backend), los cuales cada uno contempla su propio ambiente (`.env`). Se debe tener en cuenta que, al usar el servicio gratuito, los servidores tardan en levantar aproximadamente de 1 a 2 minutos después de 15 minutos sin uso.

Algo a notar, se implementó el uso de un archivo el cual describe el servicio específicamente para render (`render.yml`, `render.staging.yml`). De esta forma, podemos hacer el deploy de nuestro sistema de manera casi automática, similar a lo que es docker-compose (cabe recalcar que render no soporta docker-compose).

En el deploy se consideró suficiente usar los servidores de atlas para la persistencia de MongoDB, debido que implementa el concepto de contenedor y sharding.

5. Uso de la Inteligencia Artificial

En la mayoría de los casos se utilizó [claude.ai](#). Luego, se empezó a aplicar el uso de opencode, con modelos gratuitos, que permitió un mejor análisis al tener acceso a todos los archivos del repositorio.

5.1 Creación de test según casos de uso

Se usó IA para generar casos de prueba a partir de los casos de uso del sistema. Los tests cubrieron las tres capas de la aplicación: repositorios, servicios y controladores. En los controladores se validaron los endpoints de la API REST (respuestas esperadas, códigos HTTP y manejo de errores), en los servicios se probó la lógica de negocio del sistema de figuritas (como la verificación de duplicados, los intercambios entre usuarios y la completitud del álbum), y en los repositorios se verificó el acceso y la persistencia correcta de los datos. La IA ayudó a identificar escenarios límite y a escribir los tests de forma más ágil.

5.2 Diseño de las plantillas para el frontend

A partir de la descripción de las funcionalidades del sistema (User Stories), se generaron propuestas de layouts y estructuras de componentes que luego fueron adaptadas y refinadas por el equipo.

5.3 Generación de Componentes React

Durante el desarrollo del frontend se utilizó IA para generar componentes React reutilizables. Los componentes generados incluyen tanto elementos de UI genéricos (secciones, botones, modales) como componentes específicos del dominio del sistema, como la carta de figurita, la grilla del álbum y las vistas de intercambio. En todos los casos, el código generado fue revisado, adaptado e integrado al proyecto por el equipo, asegurando coherencia con la arquitectura y los estándares definidos.

5.4 Generación de Documentación

Por un lado, se generaron los JavaDocs de las clases del backend, documentando métodos, parámetros y valores de retorno a partir del código existente. Por otro lado, se usó para redactar documentación del proyecto, en apartados como:

- Infraestructura.

5.5 Debugging errores del Front y Back

Ante fallos o comportamientos inesperados, se compartía el mensaje de error junto con el fragmento de código relevante y contexto del sistema, y la IA ayudaba a identificar la causa y sugerir posibles soluciones.

5.6 Revisión de Pull Request en Github

Antes de mergear cambios, se compartía el diff para que la IA detectara posibles problemas, inconsistencias con el resto del sistema o mejoras en la implementación. Luego se empezó a aplicar el uso de un agente en opencode dedicado a la revisión de diff entre una rama y la develop.

6. Decisiones

Calificar

Actualmente, los usuarios pueden calificar una única vez por transacción exitosa (se concretó intercambio), sea subasta o intercambio.

Subasta

Las condiciones mínimas para adjudicar una figurita en subasta que definimos son figuritas solicitadas por el subastador, y una calificación mínima.

En cuanto a las ofertas, el subastador puede seleccionar (oferta ganadora temporalmente) o rechazarla, incluso seleccionar otra oferta a pesar de ya haber seleccionado una anteriormente. Mientras que la subasta no se cierre, los ofertantes pueden editar su oferta o incluso mejorarla (aunque tengan su oferta seleccionada), pero su estado de seleccionada se perderá, y pasará a pendiente nuevamente.

Figurita

Posee el valor de existentes y de reservadas. Cada vez que el usuario crea una subasta, envía una propuesta de intercambio o una oferta a subasta, se reserva una más de dicha figura involucrada. Si se cancelan, se elimina la reserva. Si se concretan, se elimina la reserva y se elimina una de las existentes.

Administrador

El administrador solo puede ver estadísticas de uso y actividad de la plataforma, y crear otros usuarios, sean de rol USUARIO o ADMIN.

Notificación

Para el requerimiento: una subasta de mi interés está por finalizar. Definimos el tiempo a menor a una hora.

Ofertar en subasta o proponer intercambio

Cuando el usuario intenta ofertar en una subasta o proponer intercambio, las figuritas para ofrecer que aparecerán son las que coincidan con las faltantes del usuario al que se le está ofertando/proponiendo.

7. Información adicional

Para encontrar otras decisiones de diseño y algunas otras particularidades más a detalle, pueden acceder a los .md de la carpeta “docs” en el repositorio.

 Arquitectura-y-Servicios.md

 CONSIGNA.md

 Diagrama de clases.jpg

 Enriquecimiento-imagenes-jugadores.md

 Manejo-de-autenticacion.md

 Manejo-de-errores.md

 RCA_SESION_IOS.md

 TELEGRAM_BOT.md

 decisiones-de-diseno-notables.md

 endpoints.md